

DTRL: Reinforcement Learning Framework for Cyber-Physical Systems with Dynamic Time Requirements

Yufeng Yang, Dacheng Shen, and Mengyu Liu

School of Engineering and Applied Sciences

Washington State University Tri-Cities

Richland, WA, USA

{yufeng.yang, dacheng.shen, mengyu.liu}@wsu.edu

Abstract—Cyber-Physical Systems (CPS) require real-time guarantees to ensure stability, safety, and performance, as delayed control decisions can lead to degraded accuracy or catastrophic failures. Reinforcement Learning (RL) has shown great potential in enabling adaptive control under uncertainty, but its deployment in CPS remains challenging due to unpredictable inference latency and stringent timing constraints. Existing approaches for meeting inference deadlines often rely on static strategies, such as pruning-based single-model worst-case inference (SWI) and multi-model switching (MMS), which reduce inference overhead but depend on optimization performed prior to deployment. Alternatively, dynamic adaptation methods adjust computation at runtime but their unpredictable inference latency and additional routing overhead make it difficult to provide timing guarantees. To overcome this limitation, we propose dynamic time reinforcement learning (DTRL), a novel RL framework that integrates early-exit neural networks into the control loop to achieve adaptive inference under dynamic time constraints. DTRL enables RL controllers to dynamically select intermediate exits for policy inference based on the available time budget, achieving a balance between control performance and inference latency. Across multiple simulated environments, test platforms, and real-robot validations, the results show that DTRL achieves strong control performance while maintaining high policy consistency, low runtime overhead, and predictable latency, demonstrating its strong potential for resource-aware and temporally predictable control in CPS.

Index Terms—Cyber-Physical Systems, Reinforcement Learning, Real-Time Systems, Adaptive Control

I. INTRODUCTION

Cyber-Physical Systems (CPS) tightly integrate computation, communication, and physical processes, where embedded computers continuously interact with the physical world through sensors and actuators [1]. In these systems, control decisions must satisfy strict real-time requirements to ensure stability, safety, and performance since delayed or unpredictable responses can degrade control quality and potentially lead to unsafe system behaviors. Modern CPS increasingly operate under complex dynamic environments where the computation time available for control may vary across control cycles due to workload contention, shared hardware resources, scheduling effects, or changing system priorities. Consequently, CPS controllers must not only achieve strong control performance,

but also adapt to dynamically changing inference budgets with predictable timing behavior.

Reinforcement learning (RL) has emerged as a promising paradigm for control in CPS, enabling adaptive decision-making in complex and uncertain environments where accurate models of system dynamics are difficult to obtain [2], [3]. By interacting with the system or its high-fidelity simulation, an RL controller can learn optimal or near-optimal policies that balance performance objectives such as stability, energy efficiency, and safety. However, deploying RL in real-time CPS remains challenging because policy inference must satisfy strict timing constraints. In practical deployments, each control decision must be generated within a bounded and potentially time-varying inference budget to maintain closed-loop stability and safe system operation.

A straightforward way to satisfy timing constraints is to reduce the inference cost of the policy network in the post-training stage. Existing methods can be broadly categorized into static model compression and runtime-adaptive inference. Static approaches such as pruning and quantization [4], [5], compress the policy offline to ensure that inference can meet worst-case timing requirements using a single lightweight model. Although these methods provide stable and predictable latency, they often sacrifice representational capacity and control performance, since the same compressed policy must be used regardless of the available computation time. In contrast, runtime-adaptive methods [6]–[8] dynamically adjust the amount of computation during inference, enabling the policy to trade computation for performance based on current conditions. However, these methods fail to jointly satisfy the two fundamental requirements of CPS control under dynamic deadlines: adaptability to varying computation budgets and predictability of inference latency. Consequently, there exists a critical problem in current approaches, as none can simultaneously provide both flexible computation scaling and assured timing guarantees required for safe and reliable CPS RL-enabled controllers.

To address this problem, we propose dynamic time reinforcement learning (DTRL), a novel RL framework for CPS with dynamic time requirements. Specifically, we integrate

TABLE I: Comparison between DTRL and representative inference paradigms for dynamic-deadline CPS.

Methods	Policy consistency	Low extra overhead	Latency predictability	Strong control performance
MMS	×	✓	✓	△
SWI	✓	✓	✓	×
Dynamic pruning	△	×	×	△
DTRL (ours)	✓	✓	✓	✓

Note: Multi-model switching (MMS) means selecting a model from multiple trained policies at runtime, and single-model worst-case inference (SWI) means using one compressed policy that satisfies the tightest inference budget. ✓ indicates that the property is generally preserved, × indicates that the property is not preserved, and △ indicates a partial or limited advantage.

an early-exit architecture [9] with RL, enabling the agent to dynamically adjust its inference overhead according to the available computation time while maintaining output precision. Table I shows the comparison between DTRL and representative inference paradigms. Unlike static compression methods that always use the same lightweight policy, DTRL can exploit additional computation time to improve control quality when looser deadlines are available. Unlike existing runtime-adaptive methods that rely on input-dependent routing or dynamic sub-network construction, DTRL uses predefined exits with profiled inference latency, enabling deadline-aware policy adaptation with low runtime overhead and predictable inference behavior. Our main contributions can be summarized as follows:

- We propose DTRL, an RL framework for CPS operating under dynamic real-time constraints. By integrating early-exit policy networks into the control loop, DTRL dynamically selects the deepest feasible policy exit under the available inference budget. The framework is compatible with both on-policy and off-policy RL algorithms and is independent of specific reward designs or environment structures.
- We design RL training strategies for early-exit policies to improve cross-exit action consistency. For on-policy RL, DTRL adopts multi-stage training on a shared backbone to reduce trajectory-policy mismatch. For off-policy RL, DTRL uses joint-loss training with shared replay data and critic supervision to align different exits. By reducing abrupt action changes during runtime exit switching, DTRL improves control stability and task performance under dynamic deadlines compared to conventional multi-layer designs.
- We implement and evaluate DTRL in several simulation environments and a quadrupedal testbed. The hardware experiments further demonstrate the practical feasibility of deploying DTRL on real CPS platforms.

II. RELATED WORK

A. Static Budget-Fitting Methods

A direct way to satisfy inference deadlines is to compress the policy before deployment to obtain a lightweight model that fits the worst-case timing requirement. In this work, we refer to this paradigm as SWI. Static compression methods,

such as pruning and quantization, reduce inference latency offline while keeping the model structure fixed during runtime [10]. Since the deployed architecture does not change during execution, SWI provides relatively stable and predictable latency behavior, making it suitable for real-time CPS deployment [11].

However, this predictability comes at the cost of runtime adaptability. Because the policy is compressed to satisfy the tightest deadline, the same lightweight model must be used regardless of the available computation budget. As a result, SWI may sacrifice representational capacity and control performance, and cannot exploit additional computation time when looser deadlines are available [4], [5].

B. Runtime-Adaptive Inference Methods

To improve adaptability beyond SWI, another line of work adjusts inference computation at runtime according to the current resource condition or input. A straightforward strategy is MMS, which maintains a pool of pre-trained models with different capacities and selects one of them during deployment based on the available budget or system load [12], [13]. Compared with static pruning, MMS provides greater flexibility because it can switch to a larger model when more computation time is available and fall back to a smaller one under tighter deadlines. However, this strategy requires storing multiple full models, leading to a super-linear parameter increase and additional system overhead. For resource-constrained CPS platforms, such as embedded controllers, mobile robots, and edge devices, the memory and storage overhead introduced by maintaining a pool of models can become a practical limitation.

Another class of runtime-adaptive methods, dynamic neural networks, dynamically constructs or selects a sub-network during inference [6]. These methods activate only a subset of channels [7], neurons [8], layers [14], or blocks [15] according to the current input or system state. However, they are less naturally aligned with the dynamic-deadline control problem considered in this work. First, generating or selecting a sub-network usually requires additional gating, routing, mask-generation, or reconfiguration operations. The time spent on this decision process consumes part of the limited computation budget that could instead be used for policy inference. Second, the selected execution path is primarily determined by the input, rather than directly by the currently available deadline [6], [7]. As a result, two states on the same deadline may trigger different execution paths and latencies, while the same state on different deadlines may not produce the desired computation scaling. Therefore, while dynamic neural networks represent an important class of adaptive inference methods, they do not directly address the requirement of temporally predictable computation under dynamic real-time constraints considered in this work.

C. Early-Exit Neural Networks

Early-exit neural networks (EENNs) provide a structured form of adaptive inference by inserting multiple intermediate

exits along the depth of a full neural network [9]. Unlike conventional adaptive-computation methods that dynamically construct inference paths at runtime, EENNs attach side branch classifiers at each exit and can output an intermediate prediction before reaching the final layer [16]. Although this mechanism was originally designed for efficient use of computational resources rather than deadline-aware control [17], the predefined exits naturally provide discrete computation levels that can be selected according to a runtime budget. Since each exit corresponds to a predefined computation depth, its latency can be profiled before deployment. This makes EENNs attractive for dynamic-deadline settings, where the controller must select an inference path according to the currently available time budget while maintaining predictable latency behavior [18], [19]. Therefore, EENNs provide a promising architectural foundation for deadline-aware adaptive control, where different exits can be selected according to runtime timing requirements.

III. PRELIMINARIES

This section introduces the notation and problem setting used throughout this work. After introducing the basic concepts of EENNs and RL in CPS, we define the dynamic inference deadline setting considered in this work. Based on this setting, we finally formulate our problem into two optimization problems for the training and deployment stage.

A. Early-exit Architecture

Early-Exit architecture extends neural networks by inserting additional side branches into the backbone network. As shown

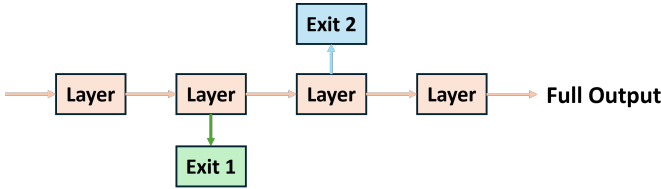


Fig. 1: Model structure for early-exit neural networks.

in Figure 1, all exits in an EENN share a common backbone, while each early exit branch produces an output based on the intermediate feature representation extracted up to its corresponding layer. Compared with multi-model switching methods that maintain several independent networks, this shared-backbone structure reduces storage overhead and provides multiple predefined inference depths within a single model. The complete set of trainable parameters in an EENN can be represented as $\Theta = \{\theta_b, \theta_1, \theta_2, \dots, \theta_N\}$, where θ_b denotes the shared backbone parameters and θ_i represents the parameters of the i -th side branch. Given an input x , the intermediate feature at the i -th exit is denoted by $h_i = f_i(x; \theta_b)$, where f_i represents the partial forward mapping of the backbone before that exit. The output of the i -th exit is then given by $\hat{y}_i = g_i(h_i; \theta_i)$ where g_i denotes the corresponding exit head.

EENNs were originally introduced for supervised learning tasks, especially image classification, where all exits can be

optimized using the same ground-truth label [9], [16]. A common supervised objective is to aggregate the losses from all exits:

$$L_{\text{EENN}} = \sum_{i=1}^N w_i L_i(\hat{y}_i, y), \quad (1)$$

where $L_i(\hat{y}_i, y)$ is the supervised loss of the i -th exit and w_i is its weighting coefficient. This formulation allows all exit heads and the shared backbone to be trained at the same time.

B. Reinforcement Learning

In RL, an agent interacts with the environment in a trial-and-error manner to learn an optimal policy that maximizes the expected cumulative reward [20]. At each time step, the agent observes the system state s_t , takes an action a_t according to the policy $\pi_\theta(a_t|s_t)$, and receives a scalar reward r_t from the environment. The policy parameters θ are iteratively updated to improve the expected return based on the feedback obtained from environment interactions. In practice, the policy can be parameterized by a neural network with parameters θ , and optimized by gradient-based methods to maximize the expected return:

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

where $\gamma \in (0, 1]$ is the discount factor.

RL has shown strong potential for CPS control because it can learn adaptive decision-making policies in complex environments where accurate system models are difficult to obtain. However, unlike conventional RL settings, CPS deployment additionally requires each policy inference to satisfy strict timing constraints in order to maintain stable and safe closed-loop operation. In practice, larger policy networks may improve control performance but also incur higher inference latency, creating a trade-off between decision quality and real-time feasibility.

RL algorithms are commonly categorized into on-policy and off-policy methods. On-policy methods update the policy using trajectories collected by the current policy, while off-policy methods can reuse data collected by previous or different policies, typically through a replay buffer. This distinction is also important for DTRL, and Section V introduces separate DTRL training strategies for them.

C. Dynamic Inference Deadline Setting

We consider a time-critical CPS using a neural-network policy once per control step. The control loop runs periodically with a fixed sampling period T_c , which is determined by the control frequency of the system. Each policy invocation is treated as an inference job that must generate an action before its deadline.

As illustrated in Figure 2, we denote the beginning of the i -th control period as s_i . The policy network starts inference on the target hardware at the beginning of this control step and ends inference at time e_i . We define the on-hardware inference time for this job as $\ell_i \triangleq e_i - s_i$. Each inference instance has

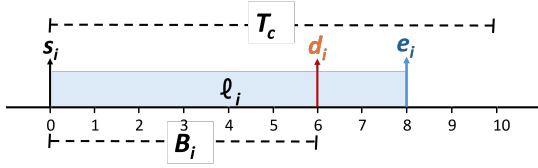


Fig. 2: Timing model of an inference job in a periodic CPS control loop ($f = 100 \text{ Hz}$, $T_c = 10 \text{ ms}$). This job misses its deadline as $e_i > d_i$, equivalently with $\ell_i > B_i$.

a real-time deadline d_i . Given the start time s_i , we define the dynamic execution-time budget for this inference job as $B_i \triangleq d_i - s_i$. The available budget B_i can vary between instances because the deadline d_i may change across control steps according to different real-time requirements, workload conditions, or available decision budgets [21]. Meanwhile, the actual inference time ℓ_i can also vary because the completion time e_i depends on the current hardware working condition, scheduling effects, and other sources of jitter during the inference time [22]–[24]. Therefore, in this work, both the available budget B_i and the realized on-hardware inference time ℓ_i are dynamic during deployment. One control step meets its deadline if and only if $e_i \leq d_i$, or equivalently, $\ell_i \leq B_i$. We do not explicitly model data transfer, pre-processing, or actuator communication delays in this formulation. Instead, these system-level delays are assumed to be reflected in the available inference budget B_i . Therefore, the inference budget B_i is generally less than or equal to control period T_c , since only part of the control period can be allocated to policy inference after accounting for other system-level operations.

In this work, the dynamic inference deadline d_i is assumed to be known or estimated before each control step. How d_i is estimated or scheduled is beyond the scope of this paper. DTRL aims to select the deepest feasible exit with the on hardware inference time ℓ_i that does not exceed the available budget B_i , while maintaining control performance.

IV. DTRL FRAMEWORK

This subsection introduces the architecture of the proposed DTRL framework. As illustrated in Fig. 3, the framework consists of three tightly integrated components: a training

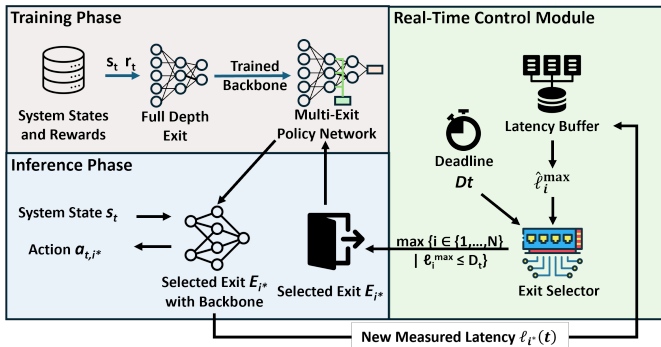


Fig. 3: Overall architecture of DTRL framework

phase, an inference phase, and a real-time module that governs deadline-aware exit selection. The training phase trains a multi-exit policy network from system states, control inputs, and rewards collected through environment interactions. DTRL adopts two training strategies to account for the different trajectory collection and policy update mechanisms of the on-policy and off-policy RL algorithms. For on-policy RL algorithms, DTRL adopts a multi-stage training method. For off-policy methods, DTRL performs joint-loss training over the shared backbone and multiple exits. Despite these differences in training, both strategies train the same early-exit policy structure, where multiple exit heads are attached to a shared backbone at different depths. These training strategies are described in detail in Section V.

The second component is the real-time control module. At each control step t , it receives the runtime deadline d_t , derives the available inference budget B_t , and maintains an online latency buffer to estimate the runtime cost $\hat{\ell}_i^{\max}$. The estimated cost $\hat{\ell}_i^{\max}$ of each exit E_i is initialized using the results of the target-hardware profiling and subsequently updated following new runtime measurements are added to the latency buffer. Here, $\hat{\ell}_i^{\max}$ is a profiling-based estimate used for runtime exit selection rather than a formally verified worst-case execution-time (WCET) bound. Based on B_t and these latency estimates, the module selects the deepest feasible exit.

The third component is the online inference phase. After the real-time control module selects the exit index $i^*(t)$, the current state of the system s_t is processed by the shared backbone up to the selected exit $E_{i^*(t)}$. The corresponding exit head then generates the control action $a_{t,i^*(t)}$ for the current control step. After the action is produced, the measured inference latency is fed back to the real-time control module to update the online latency buffer. This closes the loop between deadline-aware exit selection, policy inference, and runtime latency monitoring. The detailed online inference procedure is described in Section VI.

V. DTRL TRAINING

This section describes how DTRL trains the shared-backbone multi-exit policy. Training this policy must consider the different data-collection and update mechanisms of on-policy and off-policy RL, as well as the action inconsistency that may arise among exits. Consequently, DTRL uses separate training strategies for the two algorithm classes. We refer to the on-policy DTRL variant as DTRL-On and the off-policy DTRL variant as DTRL-Off.

A. Multi-Stage Training for On-Policy RL

Previous studies on EENNs primarily focused on supervised learning tasks, particularly in the field of computer vision [16]. In supervised learning, all branches are optimized toward the same loss with a single label. Therefore, their losses can be fused through weighted aggregation or hierarchical distillation, ensuring consistent gradient flow throughout the shared backbone [25]–[27].

However, directly training all exits together is problematic for on-policy RL. On-policy RL updates a policy using trajectories collected by the same policy that is being optimized [28]. At each control step, the policy observes the current state s_t , samples an action a_t , and the environment then returns the next state s_{t+1} and reward r_t according to the executed action. The update of a policy is expected to use trajectories sampled from the same policy. Therefore, the collected trajectory is directly coupled with the action actually taken by the policy.

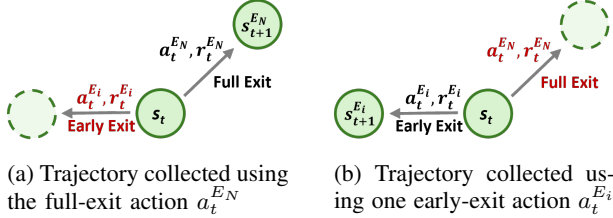


Fig. 4: Examples of reward assignment mismatch in on-policy RL when different exits suggest different actions for the same state, but the collected trajectory and reward correspond to only one executed action.

This property may generate a training conflict when an early-exit policy network is trained with multiple exits in the same time. Fig. 4 illustrates this conflict with a single state transition example during trajectory collection. At state s_t , different exits suggest different actions. One early exit E_i suggests a leftward action $a_t^{E_i}$ and the full exit suggests an upper right action $a_t^{E_N}$. However, in on-policy RL, only one action can and must be selected for trajectory collection and environment transition at each step. If the full-exit action $a_t^{E_N}$ is selected, the agent moves upper right, reaches state $s_{t+1}^{E_N}$, and receives reward $r_t^{E_N}$. However, for one early exit E_i , the same trajectory is also used to update its parameters even though its proposed action $a_t^{E_i}$ should obtain reward $r_t^{E_i}$ and reach state $s_{t+1}^{E_i}$. In other words, the early-exit action $a_t^{E_i}$ is incorrectly associated with the transition and reward produced by $a_t^{E_N}$. Conversely, if $a_t^{E_i}$ is selected for execution, all other non-executed exits may also be penalized for outcomes that they did not cause and reach state $s_{t+1}^{E_i}$. In either case, assigning the same return to all exits breaks the correspondence between each exit's proposed action and the trajectory-level outcome used for optimization. This misleading return assignment can make naive joint on-policy training unstable.

To address this problem, DTRL adopts a multi-stage training strategy for on-policy RL algorithms. Initially, only the final executable policy π_{Θ_N} is active, where $\Theta_N = \{\theta_b, \theta_N^{\text{head}}\}$. Once the full-depth policy has converged, we freeze the shared backbone parameters θ_b and attach early-exit heads to selected intermediate layers of the trained backbone. Each early exit E_i , where $i < N$, defines an executable policy π_{Θ_i} with parameters $\Theta_i = \{\theta_{b,1:m_i}, \theta_i^{\text{head}}\}$, where $\theta_{b,1:m_i}$ denotes the truncated backbone segment used by exit E_i , and θ_i^{head} denotes the parameters of the corresponding exit head.

The early exits are then trained sequentially. For each early exit E_i , the policy keeps the shared backbone θ_b and the

already trained exit heads fixed, and optimizes only the current exit-head parameters θ_i^{head} . During this stage, trajectories are collected using the active policy head π_{Θ_i} , to ensure the actions, rewards, and returns used for the update are generated by the same policy being optimized. After E_i converges, its exit-head parameters are fixed, and the next early exit is trained in the same way. This process continues until all exits have been trained.

This multi-stage design has two advantages. First, it preserves the on-policy assumption during each training stage, because the trajectories used for optimization are generated by the current active exit policy itself. Second, freezing the shared backbone prevents gradients from newly added exits from disrupting the representation already learned by the full-depth policy. As a result, DTRL obtains a multi-exit policy network with a shared backbone and multiple policy heads at different inference depths, while maintaining a stable training process for on-policy RL.

B. Joint-Loss Training for Off-Policy RL

In this subsection, we describe the training strategy for off-policy RL algorithms. Unlike on-policy RL, off-policy methods update the policy using transitions sampled from a replay buffer, which means the update data do not need to be generated by the exact policy that is currently being optimized [29]. Off-policy RL is preferred when sample efficiency is critical, because it can reuse past experiences through replay buffers. It is well suited for robotics and CPS where data collection is expensive. This property allows training multiple actor exits and one shared backbone in the same time through a joint loss using the same replay buffer collected by one single behavior policy.

However, naively optimizing all exits jointly still introduces a critical challenge. In DTRL, all exits share the same backbone representation while producing executable control policies at different inference depths. Although these exits operate on the same task and state distribution, they may prefer different actions for the same state because of their different representational capacities. Consequently, their actor losses may generate conflicting update directions for the shared backbone. This issue is particularly problematic in DTRL because exits are dynamically switched online according to runtime deadlines. If different exits learn substantially inconsistent control behaviors, runtime switching may introduce abrupt changes in executed actions and degrade control stability.

To better explain this problem, consider a three-exit policy network. Let $\mathcal{L}_i$ denote the actor loss of exit E_i . Each exit loss is backpropagated through its own exit head and the backbone prefix before the exit is attached. For the first backbone part $\theta_b^{(1)}$, which is shared by all three exits, the update is jointly determined by the weighted sum of gradients from three exits:

$$\nabla_{\theta_b^{(1)}} \mathcal{L}_{\text{joint}} = w_1 \nabla_{\theta_b^{(1)}} \mathcal{L}_1 + w_2 \nabla_{\theta_b^{(1)}} \mathcal{L}_2 + w_N \nabla_{\theta_b^{(1)}} \mathcal{L}_N$$

Here, w_i controls how strongly the loss of exit E_i contributes to the shared backbone update. For deeper backbone components, only exits whose computation paths include those com-

ponents contribute gradients. Since different exits may produce different actions for the same state, their gradients can point toward conflicting optimization directions. In the early stage of training, when the shared representation has not yet stabilized, these conflicting gradients may partially cancel each other and slow convergence. More importantly, this instability does not necessarily indicate that one exit is correct while the others are incorrect. Different exits may simply correspond to different partially learned control strategies before convergence.

To address this problem, DTRL introduces a full-exit warm-up strategy before enabling joint multi-exit optimization. The key intuition is that the full-depth policy typically provides the most expressive and stable approximation of the control objective. Therefore, DTRL first anchors the shared backbone around the representation learned by the full exit before introducing shallower exits with weaker representational capacity.

During training, the full exit E_N is first used as the initial policy for environment interaction. The collected transitions are stored in a replay buffer $(s_t, a_t, r_t, s_{t+1}, \delta_t) \in \mathcal{D}$ where $\mathcal{D}$ denotes the replay buffer and δ_t indicates episode termination. At each update step, all exits are optimized using the same mini-batch sampled from $\mathcal{D}$. Since all exits are trained from shared replayed transitions rather than separately collected trajectories, DTRL can jointly optimize the entire multi-exit actor within a unified off-policy learning framework.

The multi-exit actor adopts the same shared-backbone architecture described earlier. Let N denote the number of exits, and let E_N denote the full exit. If exit E_i is attached after backbone depth d_i , its policy parameters are $\Theta_i = \{\theta_b^{(1)}, \theta_b^{(2)}, \dots, \theta_b^{(d_i)}, \theta_i^{\text{exit}}\}$ where $\theta_b^{(j)}$ denotes the j -th shared backbone component and θ_i^{exit} denotes the exit-head parameters of E_i .

To stabilize optimization, DTRL delays the introduction of early-exit losses through a warm-up schedule. Let K_{focus} denote the warm-up duration. The joint actor objective at training step k is defined as

$$L_{\text{joint}}(k) = \begin{cases} L_N, & k \leq K_{\text{focus}} \\ \sum_{i=1}^N w_i(k) L_i, & k > K_{\text{focus}} \end{cases} \quad (2)$$

Before K_{focus} , only the full-exit actor is optimized. This stage allows the shared backbone to first learn a stable control representation using the strongest policy. After the warm-up stage, the losses of early exits are gradually introduced into the joint optimization.

Since exits share different prefixes of the backbone, only exits whose computation paths include a backbone layer contribute gradients to that layer. Therefore, for $k > K_{\text{focus}}$, the gradient received by backbone component $\theta_b^{(j)}$ is

$$\nabla_{\theta_b^{(j)}} L_{\text{joint}}(k) = \sum_{i: d_i \geq j} w_i(k) \nabla_{\theta_b^{(j)}} L_i \quad (3)$$

where $i : d_i \geq j$ denotes exits whose computation paths include backbone layer j . In contrast, each exit head only receives gradients from its own actor loss.

This design provides two important advantages. First, the shared replay buffer and shared critic allow all exits to learn from a common control objective while improving sample efficiency. Second, the full-exit warm-up stabilizes early representation learning and reduces destructive gradient interference among exits. As shown later in the ablation study, this warm-up stage significantly improves convergence speed and final policy quality across all exits.

VI. REAL-TIME MODULE AND ONLINE INFERENCE

After training, DTRL enters the online deployment stage, which consists of two interacting phases: the real-time control module and the inference phase. As shown in Fig. 3, the real-time control module determines which exit should be used according to the current runtime deadline and recent latency measurements, while the inference phase executes the selected path of the multi-exit policy network to produce the action.

At each control step t , the real-time control module receives the runtime deadline d_t and derives the available inference budget B_t . In this work, d_t is treated as an external constraint provided before each control step, while how to estimate this deadline is beyond the scope of this work. The real-time control module maintains an online latency buffer $\mathcal{L}_i$ for each exit E_i , which is updated using recent runtime records. The estimated latency $\hat{\ell}_i^{\text{max}}$ is computed as the q -th percentile of the measurements in $\mathcal{L}_i$. The percentile q is configurable according to the timing requirements of the task. A larger value of q provides a more conservative latency estimate, but potentially degrades control performance as shallower exits may be selected more frequently. Based on B_t and the latency estimates, the exit selector chooses the deepest feasible exit:

$$i^*(t) = \max\{i \in \{1, \dots, N\} \mid \hat{\ell}_i^{\text{max}} \leq B_t\}$$

If no exit satisfies the constraint, the shallowest exit is selected, and this step is recorded as a potential deadline miss. The exit selector is deliberately designed to be lightweight and deterministic. It requires at most N comparisons between B_t and the stored latency estimates $\hat{\ell}_i^{\text{max}}$ without invoking a learned gating network. Its runtime overhead is therefore small and bounded, minimizing the portion of the available time budget consumed before policy inference.

The selected exit $E_{i^*(t)}$ is then passed to the inference phase. Given the current system state s_t , the multi-exit policy network is executed along the shared backbone until the selected exit is reached. After inference finishes, the newly measured latency is sent back to the real-time control module and inserted into the latency buffer of the selected exit. This feedback loop updates the latency buffer used in subsequent control steps, allowing DTRL to account for runtime jitter, scheduling effects, and workload variation.

VII. EXPERIMENTAL SETUP

A. Tasks and Environments

We evaluated DTRL on three groups of CPS control tasks with increasing levels of deadline sensitivity: a standard benchmark task, a deadline-sensitive semicircular navigation task

with two deadline sensitivity levels, and quadruped locomotion in both simulation and real-world deployment.

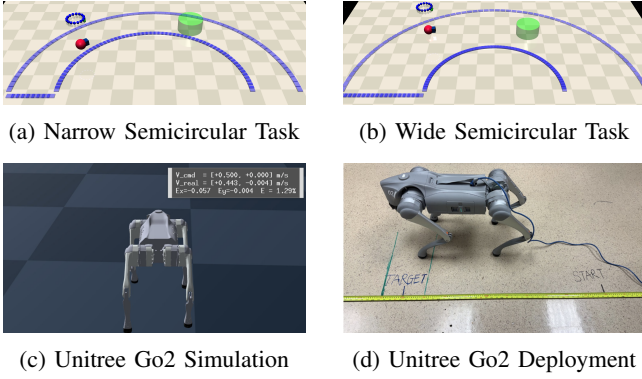


Fig. 5: Benchmarks used in the experiments. The first two tasks are custom Semicircular Reach-avoid environments, while the latter two correspond to Unitree Go2 velocity-tracking task in simulation and on real hardware.

Pendulum: We first evaluate DTRL on Pendulum, a widely used continuous-control benchmark with smooth dynamics and low-dimensional state and action spaces. Pendulum provides a controlled setting for analyzing how dynamic inference latency and deadline-aware policy selection affect control quality before evaluating more complex tasks.

Semicircular Reach-avoid: The second group of tasks consists of two customized semicircular navigation environments adapted from Safety Gym [30] to evaluate deadline-sensitive control performance. In both tasks, the agent must navigate along a curved track and reach the goal while avoiding collisions with the track boundaries. The wide-track variant provides a larger maneuvering margin and represents a moderately deadline-sensitive setting, whereas the narrow-track variant imposes tighter navigation constraints, making the task more sensitive to delayed or inconsistent control actions.

Quadruped Locomotion: We evaluate Unitree Go2 locomotion in both simulation and real-world deployment using a fixed-horizon velocity-tracking task. The robot is commanded to follow a target velocity over a fixed evaluation time. Compared with previous tasks, this experiment examines the potential of DTRL in high-dimensional robotic control and physical deployment.

B. Baselines

We compare DTRL to the following baselines under dynamic time requirements:

Full Model: The full-depth policy is always used for inference. Although it can provide strong control performance when the given computation time is sufficient, it may miss deadlines under tight runtime budgets. In this work, this baseline is implemented as a separately trained full-depth policy with the same backbone structure as the final exit of DTRL, but without early exits or deadline-aware exit selection.

Single-model Worst-case Inference: This baseline represents a classic static model-compression strategy for reducing deadline misses. It compresses a full-depth policy into a lightweight policy whose inference latency can satisfy the tightest deadline in each experiment. We obtain the lightweight policy by applying pruning followed by fine-tuning to the separately trained full-depth policy.

Multi-model Switching: This baseline represents a runtime model-selection strategy that uses multiple independent policies with different computational costs. In this work, for experimental fairness, we align the available computational levels of the MMS baseline with the exit depths used by DTRL. Specifically, MMS trains one independent policy for each DTRL exit depth to build a model pool. During evaluation, the model pool is selected according to the current runtime budget, and the largest policy whose profiled latency fits the budget is used for inference. Unlike DTRL, the policies in MMS are trained separately and do not share any common backbone.

C. Training and Evaluation Setup

As discussed in Section V, DTRL supports both on-policy and off-policy RL through different training strategies. To evaluate its applicability across RL paradigms, we use Proximal Policy Optimization (PPO) [31] as the representative on-policy algorithm and Soft Actor-Critic (SAC) [32] as the representative off-policy algorithm. For each task, all compared methods use the same random seeds, network structures, and task-specific hyperparameters for fair comparison.

This work focuses on deadline-aware policy inference rather than deadline estimation. Therefore, the dynamic deadline is treated as an external constraint provided before each control step. Before evaluation, we profile the inference latency on the target hardware over 2000 control steps. Following the SWI design goal of satisfying the tightest runtime budget, $B_{\min}$ is set to the 95th-percentile latency of SWI, while $B_{\max}$ is set to the 95th-percentile latency of the Full model. During evaluation, the available inference budget at each control step is uniformly sampled from $[B_{\min}, B_{\max}]$. These profiling

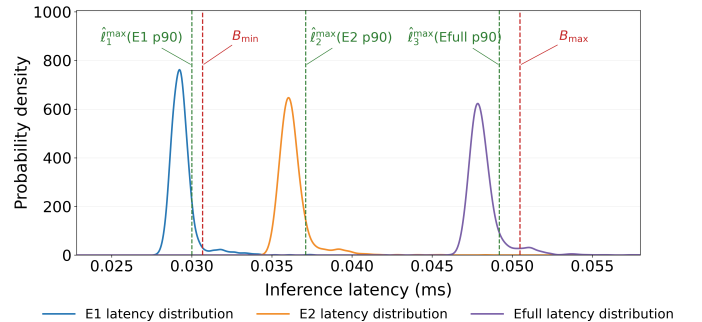


Fig. 6: Inference-latency distributions of the three DTRL-Off exits on Pendulum. The red dashed lines indicate $B_{\min}$ and $B_{\max}$, while the green dashed lines indicate latency estimates $\hat{l}_i^{\max}$ used for exit selection.

measurements are also used to initialize the latency buffer and

the resulting latency estimate $\hat{\ell}_i^{\max}$ for each exit. As described in Section VI, these estimates are updated as new runtime measurements are added to the latency buffer.

Figure 6 shows a representative runtime example on Pendulum. In this experiment, we set q , described in Section VI, to 90, which means $\hat{\ell}_i^{\max}$ corresponds to the 90-th percentile of the recent latency measurements for exit E_i . Under the latency estimates shown in Figure 6, E_1 is selected when $B_{\min} \leq B_t < \hat{\ell}_2^{\max}$, E_2 is selected when $\hat{\ell}_2^{\max} \leq B_t < \hat{\ell}_3^{\max}$, and E_3 is selected when $\hat{\ell}_3^{\max} \leq B_t \leq B_{\max}$.

Except for the real-world Unitree Go2 deployment, all simulation experiments are evaluated over 100 episodes. All experiments are conducted on a workstation with an Intel i9-14900KF CPU and an NVIDIA RTX 4080S GPU. For fairness, all latency profiling and evaluation experiments are restricted to the third CPU core.

D. Evaluation Metrics

1) *Deadline Hit Rate*: This metric measures the percentage of control steps whose inference latency does not exceed the dynamic time requirement. At control step t , let ℓ_t denote the measured inference latency and B_t denote the inference budget. The deadline hit rate is computed as $\text{HitRate} = \frac{1}{T} \sum_{t=1}^T \mathbf{1}[\ell_t \leq B_t]$ where T is the total number of control steps in one episode and $\mathbf{1}[\cdot]$ is the indicator function.

2) *Average Episode Return*: The average episode return is used to measure the overall control performance of each method. For one evaluation episode, the return is computed as $G = \sum_{t=0}^{T-1} r_t$ where r_t is the reward at the control step t , and T is the length of the episode.

3) *Task Success Rate*: Although the average episode return reflects the cumulative reward obtained by a policy, it does not always provide a direct or robust indication of whether the task objective is achieved. This is because the average return is affected not only by the general control performance, but also by the reward design, the initial state distribution, and other random settings.

Therefore, we define a task-specific success criterion for each environment and report the task success rate $\text{SuccessRate} = \frac{1}{N} \sum_{j=1}^N \mathbf{1}[\text{Success}_j]$ where N is the number of evaluation episodes or trials.

For each task, the success condition is defined separately according to the task goal. For Pendulum, success is defined as maintaining upright stabilization. An episode is considered successful if the pendulum remains close to the upright position with low angular velocity during the second half of the episode $\sum_{t=T/2}^T \mathbf{1}[\theta_t \leq \epsilon_\theta \wedge |\dot{\theta}_t| \leq \epsilon_{\dot{\theta}}] = \frac{T}{2}$ where θ_t and $\dot{\theta}_t$ denote the angle error and angular velocity at the control step t , $\epsilon_\theta = 0.02\pi$ corresponds to 3.6° , and $\epsilon_{\dot{\theta}} = 1.0 \text{ rad/s}$. For the semicircular tasks, success requires reaching the goal without colliding with the track boundary. For Unitree Go2 velocity tracking, success requires stable locomotion without falls or safety stops while maintaining acceptable velocity-tracking accuracy.

For semicircular reach-avoid tasks, an episode is considered successful if the agent reaches the goal within 1000 control

steps without colliding with the lane boundary. Both the wide and narrow track variants use the same success definition. For Unitree Go2 velocity-tracking, an episode is considered successful if the robot remains stable and tracks the commanded forward velocity within the allowed tolerance. Following the evaluation metric used by Ding et al. [33], let v_{cmd} denote the commanded forward velocity and v_t denote the measured forward velocity at control step t . The velocity-tracking error is computed as the root-mean-square normalized error:

$$E_v = \sqrt{\frac{1}{T} \sum_{t=1}^T \left(\frac{v_t - v_{\text{cmd}}}{|v_{\text{cmd}}|} \right)^2}$$

A simulated episode is counted as successful if the robot does not fall or trigger a safety stop, and $E_v \leq 0.1$. For real-world deployment, step-wise velocity measurements may be less reliable or unavailable. Therefore, we compute the average forward velocity from the measured displacement over the evaluation horizon: $\bar{v} = \frac{\Delta x}{T}$, where Δx is the measured forward displacement and T is the evaluation time. A real-world trial is counted as successful if the robot does not fall or trigger a safety stop, and the normalized average velocity-tracking error satisfies $\frac{|\bar{v} - v_{\text{cmd}}|}{|v_{\text{cmd}}|} \leq 0.1$. The 10% threshold is used as a task-specific tolerance for acceptable velocity tracking in our evaluation.

4) *Cross-Exit Action Inconsistency*: As discussed in Section V, different models in MMS or exits in DTRL may produce different actions for the same state. However, in the setting of dynamic deadlines, if different exits or independently trained models produce largely different actions for the same state, runtime switching can introduce abrupt changes in the executed control command. Such discontinuities may cause oscillatory or zigzag behavior and eventually degrade overall control performance [34]. Inspired by the driving quality metric proposed by Chen et al. [34], we define a pairwise cross-exit action inconsistency metric across all exits or models. Given a method with N computation levels, we first collect an evaluation state set by rolling out each exit or model independently on 100 episodes: $\mathcal{S}_{\text{eval}} = \bigcup_{i=1}^N \mathcal{S}^{(i)}$, where $\mathcal{S}^{(i)}$ denotes the states visited when the i -th exit or model is used for the whole episode. For each state $s_t \in \mathcal{S}_{\text{eval}}$, we feed the same state into all exits or models and obtain the corresponding actions $\{a_t^{E1}, a_t^{E2}, \dots, a_t^{(EN)}\}$. The pairwise normalized cross-exit action inconsistency is defined as:

$$\frac{1}{|\mathcal{S}_{\text{eval}}|} \sum_{s_t \in \mathcal{S}_{\text{eval}}} \frac{2}{N(N-1)} \sum_{i < j} \frac{\|a_t^{Ei} - a_t^{Ej}\|_2}{\|a_{\text{max}} - a_{\text{min}}\|_2}$$

VIII. EVALUATION

This section evaluates whether DTRL can maintain strong control performance while satisfying dynamic inference deadlines. The evaluation is organized around three main subsections: main results, ablation study and deployment of DTRL on real testbeds.

TABLE II: Overall evaluation results under dynamic inference budgets over 100 evaluation episodes. Return, deadline hit rate, and inconsistency are reported as mean $\pm$ standard deviation when available. Higher return, higher deadline hit rate, and higher success rate indicate better performance, while lower inconsistency indicates more consistent actions across computation levels. Inconsistency denotes the normalized cross-exit or cross-model action inconsistency across computation levels.

Task	Method	Return	Deadline Hit Rate	Success Rate	Inconsistency
Pendulum	Full	-576.58 ± 305.65	0.5349 ± 0.1009	25%	—
	SWI	-281.28 ± 136.98	0.9969 ± 0.0024	42%	—
	MMS	-212.45 ± 120.85	0.9670 ± 0.0461	78%	0.2455 ± 0.1332
	DTRL-On	-158.86 ± 75.81	0.9819 ± 0.0129	100%	0.0969 ± 0.0116
	DTRL-Off	-156.59 ± 71.30	0.9703 ± 0.0129	100%	0.0541 ± 0.0070
Semicircle-Wide	Full	9.6123 ± 1.9241	0.7066 ± 0.1100	87%	—
	SWI	9.3032 ± 0.0222	0.9858 ± 0.0141	100%	—
	MMS	9.6893 ± 0.1617	0.9700 ± 0.0147	100%	0.3776 ± 0.0965
	DTRL-On	10.1738 ± 0.0031	0.9584 ± 0.0298	100%	0.1489 ± 0.0855
	DTRL-Off	9.9818 ± 0.0151	0.9662 ± 0.0160	100%	0.0616 ± 0.0346
Semicircle-Narrow	Full	9.0034 ± 2.6701	0.7005 ± 0.1067	73%	—
	SWI	6.0714 ± 0.9932	0.9885 ± 0.0107	0%	—
	MMS	9.3496 ± 1.3239	0.9814 ± 0.0114	84%	0.2460 ± 0.0980
	DTRL-On	8.0480 ± 0.6802	0.9803 ± 0.0170	100%	0.2730 ± 0.1030
	DTRL-Off	9.9992 ± 0.0065	0.9797 ± 0.0102	100%	0.0540 ± 0.0301
Unitree Go2 Sim	Full	0.5796 ± 1.1021	0.4163 ± 0.1762	0%	—
	SWI	20.3118 ± 0.8892	0.9990 ± 0.0001	39%	—
	MMS	20.6799 ± 1.0009	0.9957 ± 0.0050	15%	0.1074 ± 0.0188
	DTRL-On	19.8983 ± 1.6169	0.9928 ± 0.0063	48%	0.0539 ± 0.0158
	DTRL-Off	21.9602 ± 0.0068	0.9888 ± 0.0059	100%	0.0172 ± 0.0035

A. Main Results

Table II summarizes the main evaluation results under dynamic inference budgets. Across all evaluated tasks, DTRL achieves the best overall performance between control and timing behavior. The full model usually has insufficient deadline hit rate because it always executes the largest policy. SWI achieves high deadline hit rate by using a single compressed policy for all control steps, but this worst-case fitting strategy restricts the policy general performance even when the runtime budget is relaxed. MMS can adapt the selected model according to the time budget, but it uses independently trained policies, which can introduce greater inconsistency in actions and reduce performance during runtime switching. In contrast, DTRL keeps a single shared-backbone policy with multiple early exits and selects the deepest feasible exit according to the dynamic deadline. This allows DTRL to maintain high deadline satisfaction, better utilize relaxed time budgets, and produce more consistent actions across computation levels.

1) **Overall Performance:** Figure 7 provides a visual summary of joint timing and task performance. The desirable region is in the upper-right corner, where a method achieves both a high deadline hit rate and high task success. The full model tends to fall away from this region because it frequently misses deadlines. SWI generally achieves high deadline hit rate, but its task success can be limited when the compressed policy is not expressive enough. MMS also achieves high deadline hit rates and can maintain acceptable performance in some tasks. However, its performance remains generally lower than that of DTRL, especially as the difficulty of the task increases. In contrast, DTRL remains close to the desirable

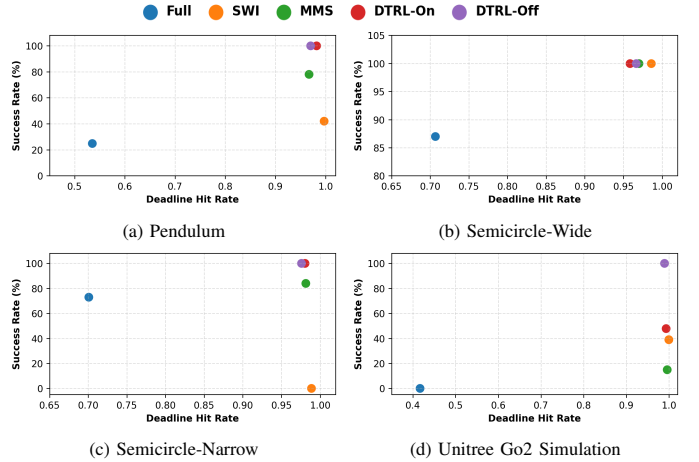


Fig. 7: Joint evaluation of deadline hit rate and task success rate under dynamic inference budgets. The upper-right region indicates better timing and task performance.

upper-right region across tasks. This suggests that DTRL achieves a better overall balance between deadline satisfaction and task performance.

2) **Importance of Deadline Satisfaction:** Deadline satisfaction is a fundamental requirement for CPS under dynamic time requirements. This can be clearly observed from the full-model baseline, which has strong control capability when sufficient inference time is available but suffers significant performance degradation once dynamic deadlines are enforced. To further highlight the impact of deadline misses, we additionally eval-

uate the same full model under a no-deadline setting. In this setting, the policy is allowed to complete inference without being constrained by the dynamic budget.

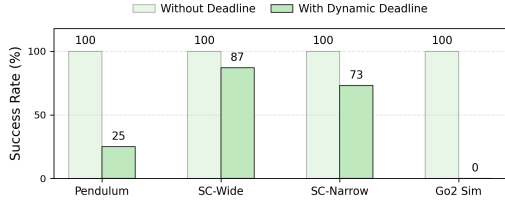


Fig. 8: Effect of dynamic deadlines on the full model. The figure compares the success rate of the same full model with and without dynamic inference deadlines over 100 episodes.

Figure 8 visualizes this performance degradation. The key observation is that the same high-capacity policy performs well when sufficient inference time is available, but degrades when dynamic deadlines are enforced. Therefore, the poor performance of the full model under dynamic deadlines is not caused by insufficient policy learning. Instead, this degradation arises from a latency-budget mismatch, showing that even a strong policy can fail when its actions cannot be produced in time. This result highlights that meeting deadline is a fundamental requirement of effective control in CPS with dynamic time requirements.

3) Effective Utilization of Dynamic Time Budgets: SWI satisfies the tightest deadlines by compressing the policy to a single lightweight model. This design is effective in meeting deadlines, but permanently limits policy capacity during deployment. This limitation becomes more visible as the task becomes more difficult. DTRL addresses this limitation by preserving multiple computation levels within one shared-backbone policy. This advantage is especially important when a lightweight model does not have enough capacity to solve the task reliably. For example, in DTRL-On, the full-depth policy is trained first and the backbone is frozen before training early exits. While this staged design preserves the on-policy assumption, shallow exits can only use early representations optimized primarily for the full-depth policy, making the earliest exit weaker than SWI in some tasks.

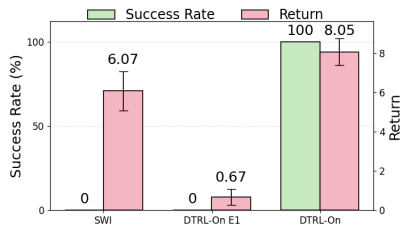


Fig. 9: Comparison between SWI, the shallowest exit of DTRL-On used alone, and full DTRL-On with dynamic exit selection on Semicircle-Narrow task.

However, unlike SWI, DTRL-On is not forced to use this shallow exit for all control steps. When the runtime budget allows, deeper exits can be selected and can guide the system

back toward successful task completion. Figure 9 illustrates this effect on Semicircle-Narrow. When only the shallowest exit of DTRL-On is used, the policy achieves a return of 0.6669 ± 0.3969 and a success rate of 0%. This result confirms that the earliest exit alone is not expressive enough to solve the narrow-track task. SWI also achieves 0% success, although its return is higher at 6.0714 ± 0.9932 , indicating that a single lightweight policy can maintain better partial control but still fails to complete the task reliably. In contrast, dynamic DTRL-On increases the return to 8.0480 ± 0.6802 and achieves a 100% success rate. This improvement shows that DTRL-On can exploit deeper exits when the runtime budget is relaxed, thereby utilizing computing resources more effectively than a single worst-case lightweight policy.

4) Action Consistency under Runtime Switching: Even when a method can automatically switch among computation levels according to the dynamic deadline, stable control also requires different computation levels to produce consistent actions for similar states. Otherwise, runtime switching may introduce abrupt command changes and disrupt the continuity of control. This problem is especially relevant to MMS in continuous-control tasks because each model is trained independently. Figure 10 shows that DTRL generally achieves

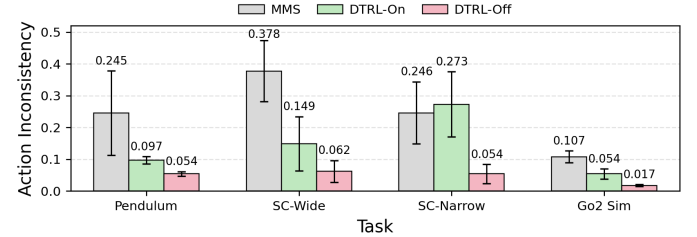


Fig. 10: Action inconsistency across computation levels for MMS, DTRL-On, and DTRL-Off. Lower values indicate more consistent actions under runtime switching.

lower action inconsistency than MMS, supporting the intuition that a shared-backbone multi-exit policy produces more aligned actions across computation levels than independently trained models. This shared-backbone design also reduces the parameter and storage overhead required to support multiple computation levels, since MMS must maintain one independent policy per level while DTRL only adds lightweight exit heads.

One exception appears on Semicircle-Narrow, where DTRL-On shows higher inconsistency than MMS. This is mainly caused by the limited capability of the earliest exit in DTRL-On. Due to the staged training procedure, the backbone is optimized primarily for the full-depth policy before being frozen for early-exit training. As a result, the earliest exit learns a conservative behavior that can avoid collisions but cannot reliably reach the goal. This is also reflected in Figure 9, where using only the earliest exit achieves low return and 0% success. However, dynamic DTRL-On can still complete the task by switching to deeper exits when the runtime budget allows.

Overall, while both MMS and DTRL adapt computation levels according to dynamic deadlines, DTRL achieves more stable task-level performance by reducing action inconsistency during runtime switching.

B. Ablation Study

After evaluating the overall effectiveness of DTRL, we further ablate two key design choices in DTRL-Off. First, we examine whether the full-exit warm-up stage is necessary for stable joint training. Second, we study how loss-weight allocation affects the performance distribution across exits.

1) **Effect of Full-Exit Warm-up in DTRL-Off:** The first ablation evaluates the effect of the full-exit warm-up strategy in DTRL-Off. As described in Section V-B, DTRL-Off first optimizes only the full exit before introducing early-exit losses into the joint optimization. To isolate the effect of this design, we compare two DTRL-Off variants with the same network architecture, training hyperparameters, replay buffer setting, and evaluation protocol. The first variant uses the proposed full-exit warm-up schedule, while the second variant removes the warm-up and optimizes all exit losses from the beginning of training. Figure 11 shows the learning

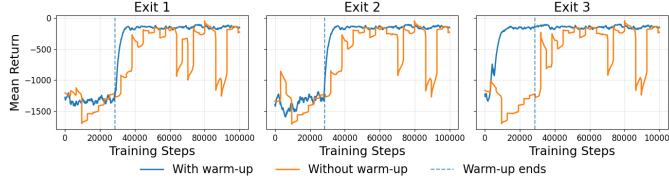


Fig. 11: Ablation of the full-exit warm-up strategy in DTRL-Off on Pendulum. Each subplot shows the evaluation return of one exit during training. The proposed warm-up strategy accelerates convergence for all exits. The dashed vertical line marks the end of the full-exit warm-up stage.

curves of the performance of the three exits under these two settings. With the proposed warm-up strategy, all exits rapidly improve after the early training stage and converge to high-return policies within fewer training steps. In contrast, when all exit losses are introduced from the beginning, the learning curves improve much more slowly for all exits, including both the early exits and the full exit. This indicates that the negative impact of starting joint optimization too early is not limited to shallow exits. Instead, losses from different exits can also slow down the learning of the full-depth policy, as early shared backbone layers are updated by potential competing learning signals before a stable control representation has been formed. Without this warm-up stage, the early shared representation may be shaped by unstable and inconsistent update signals at the beginning of training, which can further make later convergence less stable. This result supports the motivation of the full-exit warm-up strategy.

2) **Effect of Loss Weight Allocation:** The second ablation studies how the loss-weight allocation in DTRL-Off affects the performance of different early exits. In DTRL-Off, the loss weight of each exit controls how strongly this exit affects

the backbone parts included in its own computation path. We conduct this ablation on the Semicircle-Narrow task, where the shallow exit is more sensitive to the learned representation. Two weight settings are compared. The first setting assigns more weight to the full exit, with weights 2 : 3 : 5 for E_1 , E_2 , and E_3 , which is also used in our main experiment. The second setting assigns more weight to the early exits, with weights 5 : 3 : 2. All other training and evaluation settings remain unchanged. The single-exit evaluation returns over 100 episodes are summarized in Table III.

TABLE III: Ablation of loss-weight allocation in DTRL-Off on the Semicircle task. The values report the average return of each exit when evaluated independently over 100 episodes.

Loss Weights ($E_1 : E_2 : E_3$)	E_1	E_2	E_3
2 : 3 : 5	9.22	9.74	10.12
5 : 3 : 2	9.86	9.86	9.89

When the weight allocation is changed from 2 : 3 : 5 to 5 : 3 : 2, the return of E_1 improves from 9.22 to 9.86, and the return of E_2 improves from 9.74 to 9.86. This shows that increasing the weights of early exits can improve their standalone control performance. At the same time, the return of E_3 decreases from 10.12 to 9.89, indicating a trade-off between improving shallow exits and preserving the strongest full-depth performance. This design allows DTRL to choose an appropriate loss-weight combination based on the user's prior knowledge of the dynamic deadline distribution. For example, this allows the training objective to emphasize the performance of selected exits under certain dynamic deadline setting.

C. Real System Deployment

We further deploy DTRL on a real Unitree Go2 robot to evaluate whether the proposed framework can operate reliably on a physical CPS platform. The robot performs a fixed-horizon velocity-tracking task, where the average forward velocity is computed from the measured displacement over the trial duration and evaluated using the success criterion defined in Section VII-D. The full model may miss inference deadlines

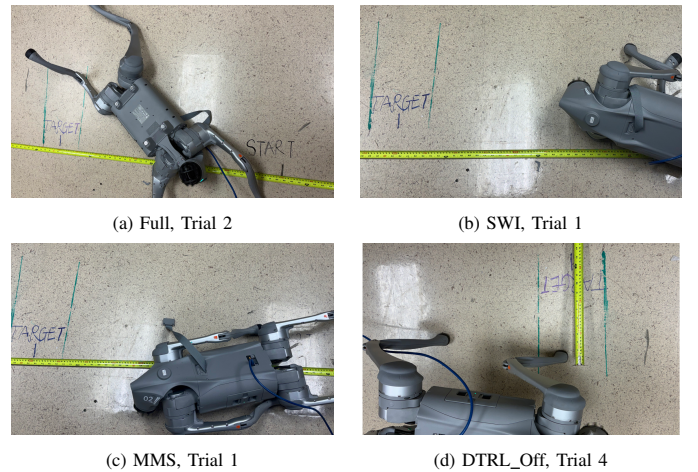


Fig. 12: Representative failures in Unitree Go2 deployment.

under dynamic runtime budgets, while SWI may produce unstable locomotion because of its limited policy capacity. Both cases can lead to falls or safety stops that may damage the hardware and interrupt subsequent experiments. Therefore, for safety reasons, we conduct only five real-world trials for each method. In addition to the velocity-tracking success criterion, we also record whether the robot falls or triggers a safety stop during each trial. This survival metric allows us to distinguish tracking failures from physically unsafe failures.

Figure 12 shows representative failure cases observed during real-world Unitree Go2 deployment. These examples are used to illustrate different failure modes rather than to summarize the overall outcome of all trials. The full model exhibits the most severe instability, which is consistent with its low deadline hit rate under dynamic inference budgets. SWI shows milder instability, suggesting that satisfying deadlines with a lightweight policy does not necessarily provide sufficient control accuracy on the physical robot. MMS can also lead to unstable behavior because independently trained policies may produce inconsistent actions when the selected model changes with the runtime budget. DTRL maintains robust locomotion and avoids the severe instability observed in the other methods.

TABLE IV: Real-world Unitree Go2 deployment results over five trials.

	Full	SWI	MMS	DTRL-On	DTRL-Off
Survival Rate	0%	0%	20%	80%	100%
Success Rate	0%	0%	0%	20%	60%

Table IV summarizes the real-world deployment results over five trials. Full and SWI fail all trials, indicating that deadline misses and insufficient policy capacity can both lead to unsafe behavior. MMS improves survival to 20% but still achieves 0% task success, consistent with its observed zigzag and intermittent motion. In contrast, DTRL-On reaches 20% success and 80% survival, while DTRL-Off achieves the best result with 60% success and 100% survival. These results show that the shared-backbone multi-exit design improves real-world stability under dynamic inference budgets.

IX. DISCUSSION

A. Loss-weight selection in DTRL-Off

As stated in section VIII-B2, the loss weights in DTRL-Off determine how strongly each exit contributes to the optimization of its shared backbone segments. As shown in Table III, assigning a larger weight to shallow exits improves their standalone performance, but may slightly reduce the performance of other exits. This trade-off suggests that there is no universally fixed optimal weight allocation. In the current implementation, these weights are manually assigned based on prior knowledge of the deadline distribution. When this distribution is known, the weights can be designed based on how frequently each exit is expected to be selected during deployment. However, the deadline distribution may be unknown or may change after deployment, making the designed

allocation suboptimal. One possible direction is to train and optimize these loss weights jointly with the policy parameters. We leave the design of deadline-aware and learnable loss-weight allocation strategies to future work.

B. Exit placement under dynamic deadlines

In the current implementation, the number and positions of exits are designed before training and remain fixed during deployment. This design assumes that users can determine the appropriate exit locations by profiling the deadline distribution in the target hardware and environment. However, if the deadline distribution shifts during deployment, the predefined exits may no longer provide latency levels that match the runtime budgets. A possible direction is to use the recent system states and runtime budgets reflected in the latency buffer to determine suitable exit locations online. DTRL could then dynamically introduce new exits at these locations during deployment, allowing the network architecture to adapt to shifts in the deadline distribution. However, how to efficiently identify and train new exits during deployment without introducing excessive or unpredictable computational overhead remains an open problem.

C. System-derived runtime budgets

In our evaluation, runtime budgets are sampled from a predefined deadline distribution devised from profiling data. Although we have already mentioned that obtaining this budget is beyond the scope of this work, the predefined budget used in our evaluation does not fully reflect how inference budgets are determined in a real-world CPS. In practice, the budget may be specified by the control task or the real-time scheduler, while the inference budget depends on the remaining time after sensing, communication, scheduling, and other system-level operations. Integrating DTRL with such a system-aware budget estimator would allow exit selection to respond to budgets determined by the actual system. How to obtain a reliable system-aware budget estimate with low extra overhead remains an important direction for future work.

X. CONCLUSION

This paper proposes DTRL, an RL framework for CPS with dynamic time requirements. By integrating early-exit policy networks into RL control, DTRL dynamically selects the deepest feasible exit under the available inference budget, enabling adaptive and predictable policy inference under dynamic deadlines. To reduce action inconsistency during runtime exit switching, we design RL-specific training strategies for both on-policy and off-policy RL. Experimental results across multiple simulation tasks and real-world Unitree Go2 deployment show that DTRL achieves a strong balance between deadline satisfaction, control performance, and action consistency, demonstrating its effectiveness for RL-enabled CPS under dynamic real-time constraints.

XI. ACKNOWLEDGMENT

This work was supported in part by Washington State University Tri-Cities new faculty seed grant and NAIRR260111.

REFERENCES

- [1] R. Rajkumar, I. Lee, L. Sha, and J. Stankovic, "Cyber-physical systems: the next computing revolution," in *Proceedings of the 47th design automation conference*, 2010, pp. 731–736.
- [2] M. Liu, P. Lu, X. Chen, O. Sokolsky, I. Lee, and F. Kong, "Model-free pac time-optimal control synthesis with reinforcement learning," in *2024 22nd ACM-IEEE International Symposium on Formal Methods and Models for System Design (MEMOCODE)*. IEEE, 2024, pp. 34–45.
- [3] —, "Deadline-safe reach-avoid control synthesis for cyber-physical systems with reinforcement learning," in *2024 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2024, pp. 96–108.
- [4] M. Shen, H. Yin, P. Molchanov, L. Mao, J. Liu, and J. M. Alvarez, "Halp: hardware-aware latency pruning," *arXiv preprint arXiv:2110.10811*, 2021.
- [5] K. Salamun, I. Pavić, H. Džapo, and I. Čuljak, "Weakly hard real-time model for control systems: A survey," *Sensors*, vol. 23, no. 10, p. 4652, 2023.
- [6] Y. Han, G. Huang, S. Song, L. Yang, H. Wang, and Y. Wang, "Dynamic neural networks: A survey," *IEEE transactions on pattern analysis and machine intelligence*, vol. 44, no. 11, pp. 7436–7456, 2021.
- [7] X. Gao, Y. Zhao, R. Mullins, C.-z. Xu *et al.*, "Dynamic channel pruning: Feature boosting and suppression," *arXiv preprint arXiv:1810.05331*, 2018.
- [8] E. Bengio, P.-L. Bacon, J. Pineau, and D. Precup, "Conditional computation in neural networks for faster models," *arXiv preprint arXiv:1511.06297*, 2015.
- [9] S. Teerapittayanon, B. McDanel, and H.-T. Kung, "Branchynet: Fast inference via early exiting from deep neural networks," in *2016 23rd international conference on pattern recognition (ICPR)*. IEEE, 2016, pp. 2464–2469.
- [10] Z. Yuan, B. Wu, G. Sun, Z. Liang, S. Zhao, and W. Bi, "S2dnas: Transforming static cnn model for dynamic inference via neural architecture search," in *European Conference on Computer Vision*. Springer, 2020, pp. 175–192.
- [11] R. Wilhelm, J. Engblom, A. Ermedahl, N. Holsti, S. Thesing, D. Whalley, G. Bernat, C. Ferdinand, R. Heckmann, T. Mitra *et al.*, "The worst-case execution-time problem—overview of methods and survey of tools," *ACM transactions on embedded computing systems (TECS)*, vol. 7, no. 3, pp. 1–53, 2008.
- [12] I. Gog, S. Kalra, P. Schafhalter, J. E. Gonzalez, and I. Stoica, "D3: a dynamic deadline-driven approach for building autonomous vehicles," in *Proceedings of the Seventeenth European Conference on Computer Systems*, 2022, pp. 453–471.
- [13] J. Zhang, S. Elnikety, S. Zazar, A. Gupta, and S. Garg, "{Model-Switching}: Dealing with fluctuating workloads in {Machine-Learning-as-a-Service} systems," in *12th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 20)*, 2020.
- [14] J. Seon, J. Hwang, J. Mun, and B. Han, "Stop or forward: Dynamic layer skipping for efficient action recognition," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2023, pp. 3361–3370.
- [15] Z. Wu, T. Nagarajan, A. Kumar, S. Rennie, L. S. Davis, K. Grauman, and R. Feris, "Blockdrop: Dynamic inference paths in residual networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 8817–8826.
- [16] H. Rahmath P, V. Srivastava, K. Chaurasia, R. G. Pacheco, and R. S. Couto, "Early-exit deep neural network-a comprehensive survey," *ACM Computing Surveys*, vol. 57, no. 3, pp. 1–37, 2024.
- [17] Y. He and L. Xiao, "Structured pruning for deep convolutional neural networks: A survey," *IEEE transactions on pattern analysis and machine intelligence*, vol. 46, no. 5, pp. 2900–2919, 2023.
- [18] K. Berestizshevsky and G. Even, "Dynamically sacrificing accuracy for reduced computation: Cascaded inference based on softmax confidence," in *International conference on artificial neural networks*. Springer, 2019, pp. 306–320.
- [19] M. Ebrahimi, A. d. S. Veith, M. Gabel, and E. de Lara, "Combining dnn partitioning and early exit," in *Proceedings of the 5th International Workshop on Edge Systems, Analytics and Networking*, 2022, pp. 25–30.
- [20] Y. Li, "Deep reinforcement learning: An overview," *arXiv preprint arXiv:1701.07274*, 2017.
- [21] M. Sudvarg, J. Buhler, R. D. Chamberlain, C. Gill, J. Buckley, and W. Chen, "Parameterized workload adaptation for fork-join tasks with dynamic workloads and deadlines," in *2023 IEEE 29th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*. IEEE, 2023, pp. 232–242.
- [22] W. Pang, X. Jiang, M. Lv, T. Gao, D. Liu, and W. Yi, "Toward the predictability of dynamic real-time dnn inference," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 9, pp. 2849–2862, 2021.
- [23] Y. Xiang and H. Kim, "Pipelined data-parallel cpu/gpu scheduling for multi-dnn real-time inference," in *2019 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2019, pp. 392–405.
- [24] Y.-H. Shin, T.-G. Song, G. Ji, and H.-W. Park, "Actuator-constrained reinforcement learning for high-speed quadrupedal locomotion," *arXiv preprint arXiv:2312.17507*, 2023.
- [25] D.-J. Han, J. Park, S. Ham, N. Lee, and J. Moon, "Improving low-latency predictions in multi-exit neural networks via block-dependent losses," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 11, pp. 16927–16935, 2023.
- [26] P. Haseena Rahmath and K. Chaurasia, "Adaptive early-exit inference in graph neural networks based hyperspectral image classification," in *International Conference on Intelligent Systems Design and Applications*. Springer, 2023, pp. 444–453.
- [27] F. Ilhan, S. F. Tekin, S. Hu, T. Huang, K.-H. Chow, and L. Liu, "Hierarchical deep neural network inference for device-edge-cloud systems," in *Companion Proceedings of the ACM Web Conference 2023*, 2023, pp. 302–305.
- [28] M. Andrychowicz, A. Raichuk, P. Stańczyk, M. Orsini, S. Girgin, R. Marinier, L. Hussenot, M. Geist, O. Pietquin, M. Michalski *et al.*, "What matters in on-policy reinforcement learning? a large-scale empirical study," *arXiv preprint arXiv:2006.05990*, 2020.
- [29] M. Hausknecht, P. Stone, and O.-p. Mc, "On-policy vs. off-policy updates for deep reinforcement learning," in *Deep reinforcement learning: frontiers and challenges, IJCAI 2016 Workshop*. AAAI Press New York, NY, USA, 2016, pp. 9–11.
- [30] J. Ji, B. Zhang, J. Zhou, X. Pan, W. Huang, R. Sun, Y. Geng, Y. Zhong, J. Dai, and Y. Yang, "Safety gymnasium: A unified safe reinforcement learning benchmark," *Advances in Neural Information Processing Systems*, vol. 36, pp. 18964–18993, 2023.
- [31] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [32] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International conference on machine learning*. Pmlr, 2018, pp. 1861–1870.
- [33] J. Ding, X. Chen, G. E. Katz, and Z. Gan, "Towards dynamic quadrupedal gaits: A symmetry-guided rl hierarchy enables free gait transitions at varying speeds," *arXiv preprint arXiv:2510.10455*, 2025.
- [34] J. Chen, D. V. Le, Y. Li, Y. Liu, and R. Tan, "Timelynet: Adaptive neural architecture for autonomous driving with dynamic deadline," *ACM Transactions on Embedded Computing Systems*, 2025.